

Docker for Beginners

A simple, story-driven guide to what Docker is, why it exists, and how it works — from history to architecture to your first container.

1. The problem that existed before Docker

Imagine you're a developer. You build an app on your laptop. It works perfectly. You send it to your teammate — and it breaks on their machine. You deploy it to a server — and it breaks there too.

Why does this happen? Because every machine is slightly different: different OS versions, different versions of Python/Node/Java, different installed libraries, different configuration and environment variables.

"But it works on my machine!" — every developer, at some point.

Every environment — your laptop (development), the testing server (staging), and the live server (production) — behaved a little differently, and those small differences caused constant, frustrating bugs.

2. The old fix: Virtual Machines

Before Docker, the standard solution was the **Virtual Machine (VM)**. A VM packages an entire operating system — kernel, drivers, everything — along with your app, so it behaves identically anywhere.

This worked, but it was heavy: each VM contains a full guest OS (several gigabytes), can take minutes to boot, and running many VMs on one server consumes huge amounts of CPU and RAM because each one duplicates the entire OS.

At the same time, software design was shifting toward **microservices** — many small independent services instead of one giant app — which made the VM overhead problem even worse.

3. Docker enters the story (2013)

Linux already had low-level features — **namespaces** (isolate what a process can see) and **cgroups** (limit CPU/memory/disk usage) — that could isolate processes without a full OS. Tools like LXC used them, but they were hard for ordinary developers to use.

A company called **dotCloud** (later renamed **Docker, Inc.**), led by **Solomon Hykes**, built a simple, friendly layer on top of these Linux features and released it as open source in 2013. It introduced the **Dockerfile**, **images**, the **Docker Hub** registry, and a simple CLI — making containers accessible to everyone, not just infrastructure teams at huge companies.

In short: Docker is an **OS-level virtualization** (containerization) platform. Instead of running a separate guest OS for every app like a VM does, containers share the host machine's OS kernel — which is exactly what makes them lightweight, fast to start, and portable, while still keeping each app isolated from the others.

4. Containers vs Virtual Machines

This diagram shows the core difference: a VM virtualizes the whole hardware/OS stack, while a container shares the host OS and only packages the application layer.

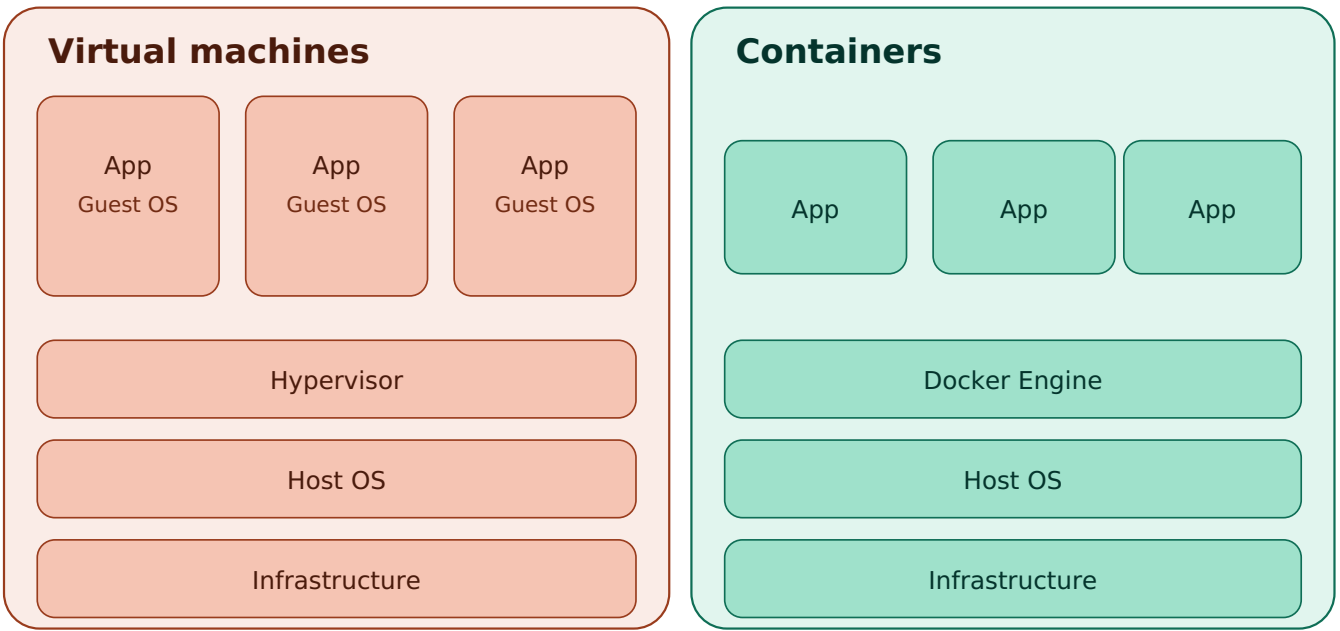


Figure 1: Virtual machines each duplicate a full guest OS. Containers share one host OS via the Docker Engine.

This is why containers start in **milliseconds** instead of minutes, and take up **megabytes** instead of gigabytes — letting you run far more of them on the same server.

What Docker's solution gives you

Quality	What it means
Portability	Runs the same on your local machine, in the cloud, or on an on-premise server.
Consistency	Identical behavior in development, testing, and production.
Lightweight	No full OS per app — containers share the host kernel.
Scalability	Ideal for microservices and orchestrators like Kubernetes and Docker Swarm.
Efficiency	Starts in seconds and uses far fewer system resources than a VM.

5. Docker architecture: how the pieces talk to each other

Docker uses a **client-server architecture**. There are three main players, and understanding how they talk to each other is the key to understanding Docker.

- The **Docker Client** is what you type commands into.
- The **Docker Daemon** is the background service that does the actual work — building images, starting containers, managing networks.
- The **Docker Registry** (like Docker Hub) is where ready-made images are stored, so the daemon can download them when needed.

The client and daemon can run on the same machine, or the client can connect to a daemon running remotely. Either way, they communicate through a **REST API**, over a UNIX socket or a network connection.



When you type "docker run", the client sends the request to the daemon, which pulls the image from the registry (if needed) and starts the container.

Figure 2: Docker's client-server architecture — client, daemon, and registry.

Zooming in a little further, here's what actually happens on the **Docker host** when you run the three most common commands — `docker build`, `docker pull`, and `docker run`:

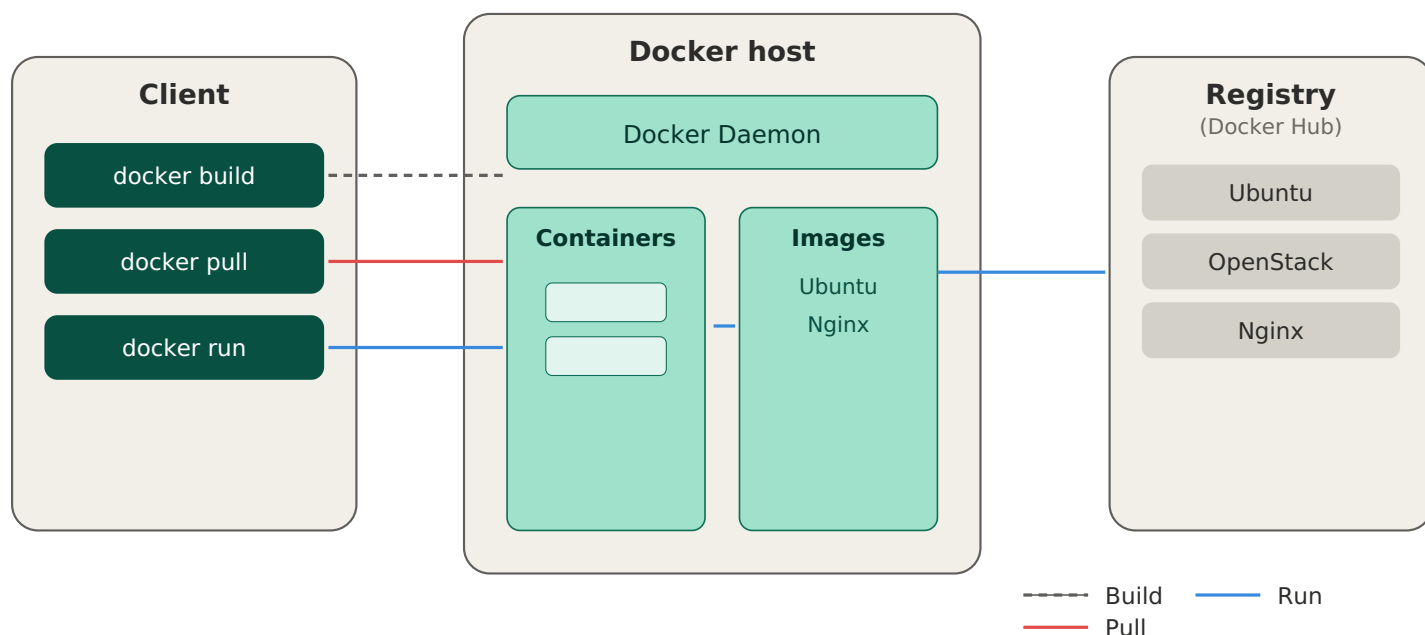
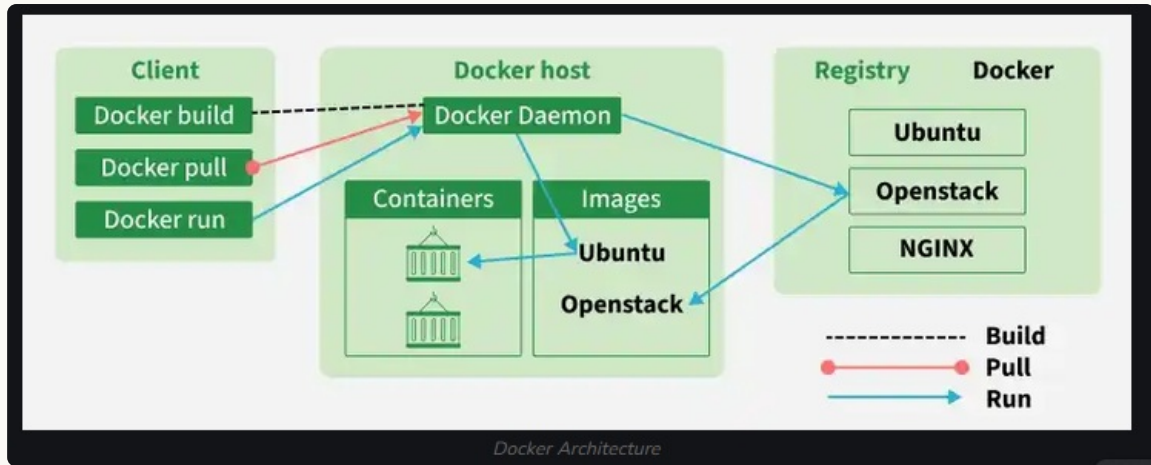


Figure 3: Docker host in detail — the daemon keeps local images and containers; build creates an image locally, pull fetches one from the registry, and run turns an image into a running container.



The original reference image this section is based on.

In plain words:

- `docker build` — the client tells the daemon to build a new image from a Dockerfile, right there on the host.
- `docker pull` — the client tells the daemon to fetch a ready-made image from the registry (Docker Hub) and store it locally.
- `docker run` — the daemon takes an image (built or pulled) and starts it as a running container.

6. The building blocks of Docker

Docker is really made up of a small set of simple pieces working together. Here's what each one does:

Component	What it is
Docker Engine	The core runtime software. It includes the daemon that actually creates and manages containers.
Dockerfile	A text file listing the steps needed to build an image.
Docker Image	A read-only template — the application code plus everything it depends on — used to create containers.
Docker Hub	A cloud-based repository for finding and sharing container images.
Docker Registry	The storage and distribution system for images, public or private.
containerd	A component inside the engine that manages each container's full lifecycle — start, stop, pause, delete.
runc	The low-level runtime that talks directly to the Linux kernel to actually create the isolated container.

Docker Engine

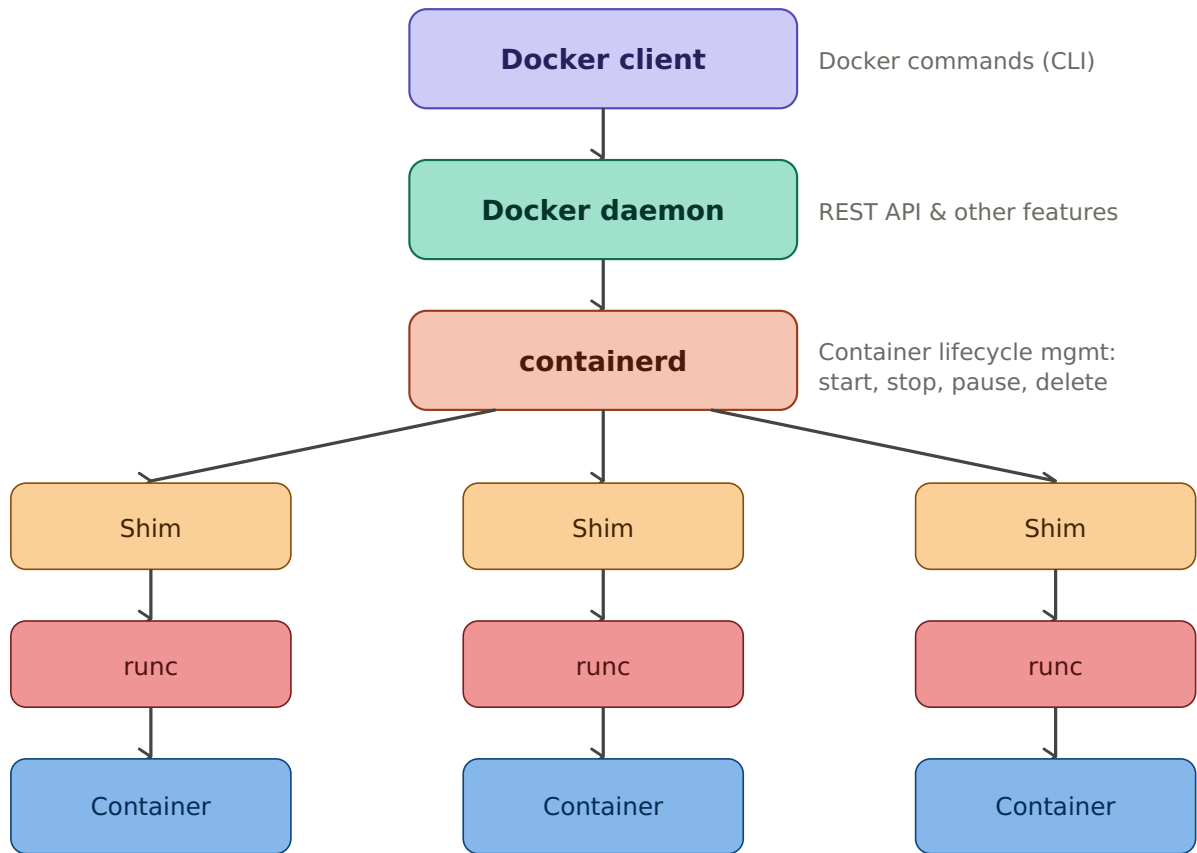
The Docker Engine is what actually makes containerization possible — without it, no image could be built and no container could run. It connects the client to the daemon:

- The **client** sends commands (`docker build`, `docker run`, etc.).
- The **daemon** receives those commands and performs the container operations.

- The **REST API** is the interface that makes this communication possible.

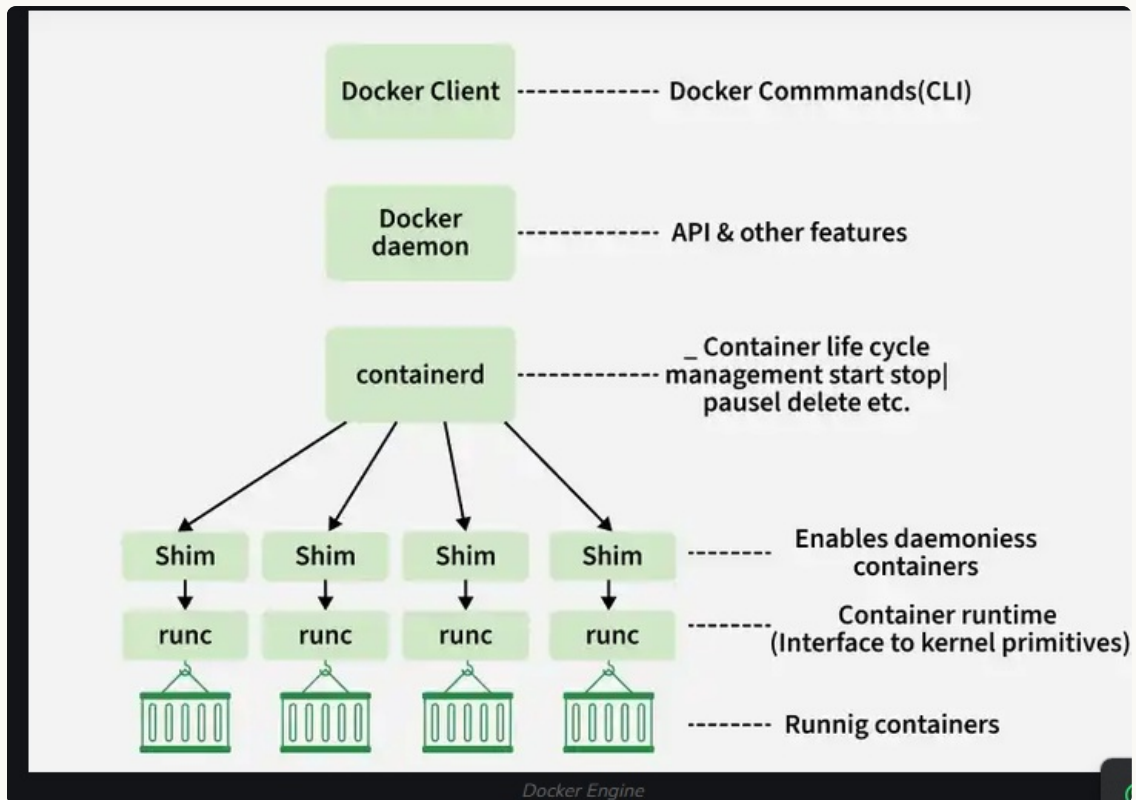
What's really inside the Docker Engine

"Docker daemon" is actually a stack of smaller programs, each handling one job. Here's the full chain from the command you type down to the actual running container:



Each shim keeps its container alive even if the daemon restarts — running independently, per container.

Figure 4: Inside the Docker Engine — the request travels from the client, through the daemon and containerd, down through a shim and runc for each container, to the actual running process.



The original reference image this section is based on.

In plain words, each layer has exactly one job:

- **Docker client** — where you type commands like `docker run`.
- **Docker daemon** — exposes the REST API and coordinates everything.
- **containerd** — manages the full lifecycle of containers: starting, stopping, pausing, deleting.
- **shim** — a small helper process, one per container, that lets containers keep running even if the daemon restarts (this is what makes containers "daemonless").
- **runc** — the low-level runtime that actually talks to the Linux kernel (namespaces and cgroups) to create the isolated container.

7. The core Docker concepts, one by one

Dockerfile — the recipe

A plain text file with step-by-step build instructions, written top to bottom, which the Docker daemon reads and executes in order — for example: start from a base Linux system, install Python, copy in the application code, and specify the command to run when the container starts.

A Dockerfile is just a list of instructions. Running it through `docker build` is what turns it into an actual image.

Docker Image — the blueprint

An image is a file made of multiple stacked layers that contains everything needed to run an application: the code, the runtime, libraries, environment variables, and configuration. Built from a Dockerfile using `docker build`, an image is:

- **Read-only** — it never changes once built.
- **Layered** — each instruction adds a cached, reusable layer, which makes rebuilding fast.
- **Portable** — it runs identically on any machine with Docker installed.

Docker Container — the live, running thing

A container is a lightweight, runnable instance of an image. It packages the application together with all its dependencies and runs it in an isolated environment, so the app behaves the same on a laptop, a test server, or in production.

- A container is created the moment an image is **run**.
- It runs as an isolated process on the host machine, but shares the host's OS kernel.
- Many containers can run side by side on the same system without interfering with each other.



Figure 5: An image is the static blueprint. A container is what you get when that blueprint is actually run.

8. How it all fits together

This diagram shows the full workflow, from writing a Dockerfile to running a container, plus sharing images through a registry.

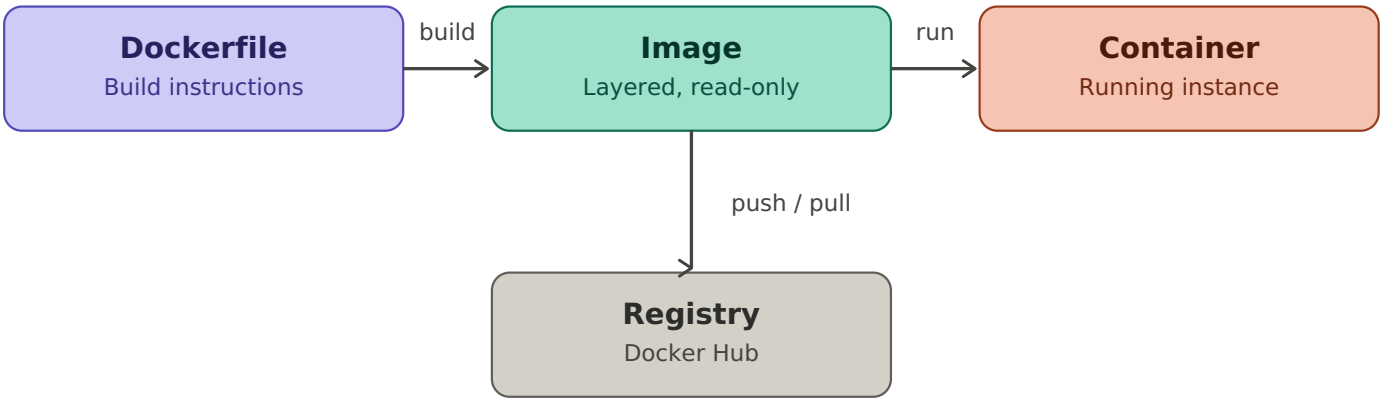
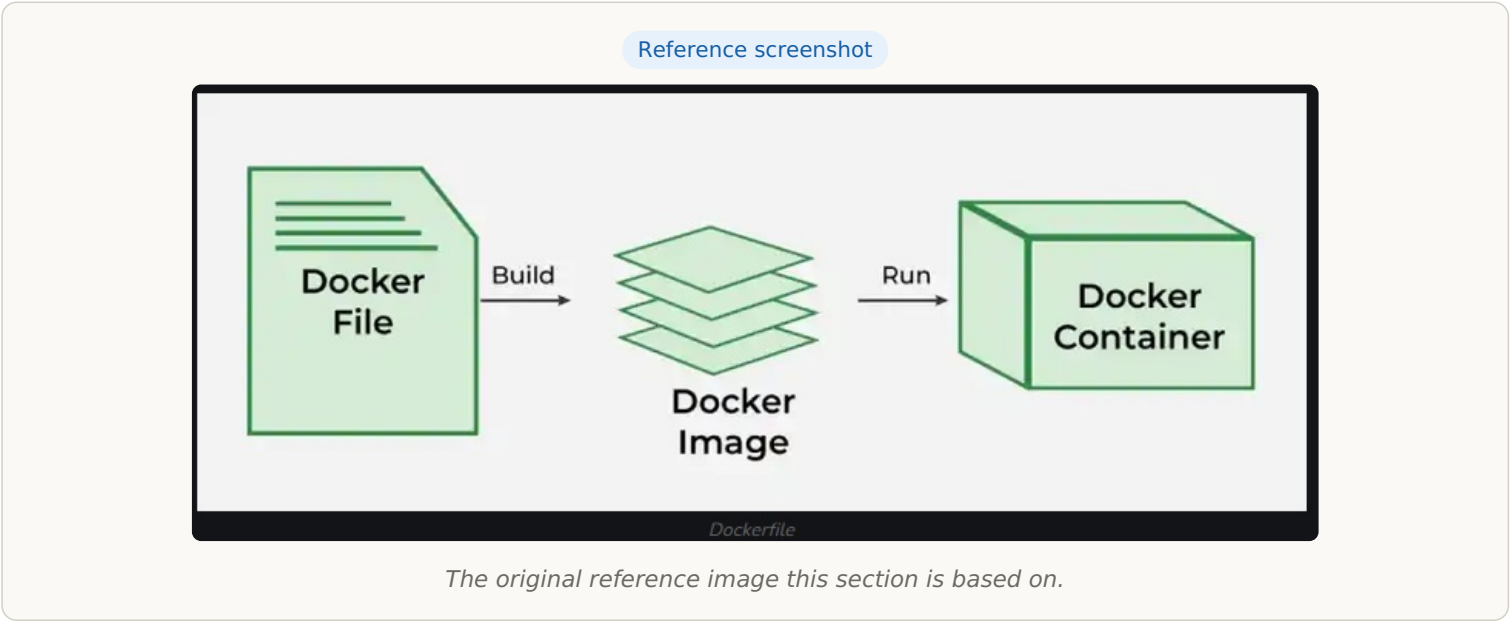


Figure 6: Dockerfile becomes an image via `docker build`; the image becomes a container via `docker run`; images are shared through a registry with push/pull.



What happens under the hood when you run `docker run`

1. You type `docker run` in the terminal — this is the **Docker Client**.
2. The client sends the request to the **Docker Daemon** (`dockerd`), running in the background, via the REST API.
3. The daemon checks if the image exists locally. If not, it **pulls** it from the registry.
4. The daemon creates a new container using **namespaces** (isolates what it can see) and **cgroups** (limits its resource usage).
5. Your app starts running inside that isolated container.

9. Why Docker actually matters

Benefit	What it means

Consistency	The exact same image runs on your laptop, in testing, and in production. No more "works on my machine."
Speed	Containers start in milliseconds, not minutes.
Density	You can run far more containers than VMs on the same hardware.
Portability	An image runs the same way on any machine with Docker installed.
Enabled microservices	Teams package each small service independently, often coordinated with Docker Compose or Kubernetes at scale.

10. Quick glossary

Term	Meaning
Dockerfile	Text file with build instructions
Image	A built, read-only snapshot of an app and its environment
Container	A running instance of an image
Docker Engine	The runtime, including the daemon, that builds and runs containers
Docker Daemon (dockerd)	The background service that manages images, containers, networks, and volumes
Docker Client (CLI)	The command-line tool you type commands into
Registry (Docker Hub)	Where images are stored and shared, publicly or privately
Namespace	Linux feature that isolates what a process can see
cgroup	Linux feature that limits resources (CPU, memory) a process can use

One-line summary: Docker is a tool that packages an application together with everything it needs to run, into a lightweight, portable "container" — so it behaves exactly the same anywhere it's deployed, without the heavy overhead of a full virtual machine.